Predictive Optimization Inventory Gate

Design & Change Specification — Deep Clone Cross-Region Migration Orchestrator

Status: Implemented & live-tested (vk-west) • Date: Oct 21, 2026 • Scope: INVENTORY + DRY_RUN modes

1. Objective

Before any table is onboarded for a Deep Clone migration, the orchestrator must verify that **Unity Catalog Predictive Optimization (PO) is disabled** for that source table. If PO is **enabled** at *any* inherited level (account→catalog→schema→table), the table must not be queued: its state is recorded in the inventory/control table and the inventory step fails. The check is **fully automatic** (driven by the input CSV, no new widgets), applies to every selection level (catalog, schema, or table rows), and runs under the **existing parallelism** model — no new sequential bottleneck.

2. Why this is needed

Predictive Optimization runs background `OPTIMIZE` / `VACUUM` operations on the source. Those commits race with the Deep Clone snapshot read and are a primary trigger of the `DELTA_CONCURRENT_WRITE` / `ConcurrentWriteException` failures seen in production chunk jobs. Gating onboarding on "PO disabled" removes that class of failure at the source, before compute is ever spent.

3. How PO is detected (API verified live first)

Before writing any code, the UC REST API behaviour was confirmed against three live fixtures in vk-west (catalog-, schema-, and table-level settings). A **single** GET per table is sufficient — Unity Catalog resolves the full inheritance chain for us:

```
GET /api/2.1/unity-catalog/tables/{catalog}.{schema}.{table}

  enable_predictive_optimization           → this object's OWN setting (ENABLE|DISABLE|INHERIT)
  effective_predictive_optimization_flag {
      value              : "ENABLE" | "DISABLE"      ← fully resolved, authoritative
      inherited_from_type : "CATALOG" | "SCHEMA" | null
      inherited_from_name : "<name of the object the setting came from>"
  }
```

> **Key simplification from the live probe:** because `effective_predictive_optimization_flag.value` already reflects inheritance, we do **not** need to walk or cache the catalog/schema tiers. One call per table — which also fits the existing per-table parallel inventory worker cleanly.

4. Decision mapping

Effective value	Outcome	Recorded
`DISABLE`	Allowed → table proceeds to `QUEUED`	normal inventory record
`ENABLE`	**Blocked**	`status=PO_BLOCKED`, `error_code=PREDICTIVE_OPTIMIZATION_ENABLED`, level + source object in message
unreadable / API error	**Blocked (fail-closed)**	`status=PO_BLOCKED`, `error_code=PREDICTIVE_OPTIMIZATION_UNKNOWN`

5. Confirmed design decisions

Decision	Choice	Rationale

Blocked-table state	New terminal status `PO_BLOCKED`	Distinct from `FAILED`; auditable, not retried, not re-dispatched.
Scope when offenders exist	**Queue the clean tables**, block only offenders, then fail the step	No partial silent success; operators see exactly what passed and what to fix.
DRY_RUN behaviour	Fail the dry run if any selection is PO-enabled	Dry run is the pre-flight; it must surface blockers before a real run.
Unreadable PO status	Fail closed (treat as blocked)	Safer to stop than to clone a table whose PO state is unknown.

6. Execution flow (what changed vs the existing procedure)

```
EXISTING inventory worker (per table, parallel)      NEW inventory worker (per table, parallel)
-----------------------------------------------      -----------------------------------------------
  resolve selection                                    resolve selection
  classify workload / size                     >>      PO CHECK  (GET effective flag)   << added, first
  upsert record as QUEUED                               if ENABLED/UNKNOWN:
                                                            mark PO_BLOCKED  (batched MERGE)
                                                            do NOT queue
                                                        else:
                                                            classify workload / size
                                                            upsert record as QUEUED
-----------------------------------------------      -----------------------------------------------
  step always succeeds if rows discovered              step FAILS if any table was PO_BLOCKED
```

Everything else — CSV/YAML resolution, chunk planning, batching, Deep Clone dispatch, validation — is **unchanged**. The gate is a new first stage inside the *same* `ThreadPoolExecutor` worker; it adds no new sequential pass and keeps the configured `inventory_parallel_threads` concurrency.

7. Components changed

File	Type	Change
`orchestrator/predictive_optimization.py`	NEW	`PredictiveOptimizationChecker` (single-table `check()`), `POState` dataclass, and `check_selections()` parallel helper for DRY_RUN. Fail-closed on error.
`orchestrator/api_client.py`	MOD	Added `unity_catalog_get(kind, full_name)` — URL-safe single-securable GET.
`orchestrator/models.py`	MOD	Added `PO_BLOCKED` to `MigrationStatus`, to `TERMINAL_STATES`, and to `VALID_TRANSITIONS` (terminal — no onward transitions, never retried).
`orchestrator/inventory_manager.py`	MOD	Worker runs PO check first; aggregation splits clean vs blocked; new `_mark_po_blocked_batch()` batched MERGE; stats gain `po_blocked` + `po_blocked_list`.
`notebooks/orchestrator_notebook.py`	MOD	Builds the checker; wires it into `InventoryManager`; DRY_RUN pre-checks & raises; INVENTORY raises `RuntimeError` after planning when `po_blocked > 0`.
`notebooks/setup_control_tables.py`	MOD	Status comment/doc updated to include `PO_BLOCKED`.
`tests/test_predictive_optimization.py`	NEW	Unit tests: flag parsing (enable/disable/inherit/missing/error), parallel `check_selections`, and `InventoryManager` integration.

8. New status lifecycle

`PO_BLOCKED` is **terminal**. A blocked table is never queued, never chunked, never dispatched, and never auto-retried. To migrate it, an operator disables PO on the source (table/schema/catalog) and re-runs inventory; the table is then re-evaluated and queued normally.

> **Operational note (unchanged by this feature):** the per-chunk worker timeout and orchestrator run timeout were separately raised to **7 days**. The stale-record reconcile threshold (`stale_threshold_minutes = 120`) remains far below that window and should be revisited to avoid re-dispatching long-running chunks — tracked separately from the PO gate.
